Applied Security Lab Write-Up

Over-Permissioned Service Principal — Application-Only Graph Access

Lab title	Over-Permissioned Service Principal: Application-Only Graph Access
Author	Kevin Mwangi
Environment	Microsoft 365 E5 trial tenant - wafulamwangi.onmicrosoft.com (isolated lab tenant, no production data)
Date	October 2026
Target platform	Microsoft Entra ID / Microsoft Graph API - application permissions
Classification	Internal lab exercise

1. Executive Summary

This lab examines the risk profile of application (app-only) Microsoft Graph permissions, as distinct from the delegated permissions covered in the earlier illicit consent grant lab. The same test application ('Expense Reporting Tool') was granted three application-level permissions, including: Mail.Read, Files.Read.All, and User.Read.All. Each required explicit admin consent. Once granted, the application authenticated using the OAuth2 client credentials flow: a direct, server-to-server exchange using only a client ID and secret, with no user, no browser, and no sign-in screen involved at any point.

The resulting access token was decoded and inspected, confirming tenant-wide, unattended read access to every mailbox, every SharePoint site collection, and every user profile in the organization. A live call against the Microsoft Graph API using this token successfully enumerated every user account in the tenant. The central finding is that application permissions carry materially higher risk than their delegated counterparts of the same name, because there is no user present to notice, be phished, or behave anomalously. The only control standing between a leaked client secret and full tenant-wide data access is the admin consent decision made once, at setup time.

2. Scenario

Application permissions are commonly granted during integration setup, connecting a SaaS tool, a reporting platform, or an automation script to Microsoft Graph as a background service. Because these integrations are typically configured once by an administrator and then left running indefinitely, the admin consent decision at setup time functions as the sole gate on an otherwise permanent, unattended credential. This lab demonstrates both the subtlety of that risk and how identical permission names behave very differently under Application versus Delegated type and its concrete impact once granted.

Critically, the same permission name does not mean the same scope of access:

- Mail.Read (Delegated) = read the signed-in user's own mailbox
- Mail.Read (Application) = read mail in all mailboxes, tenant-wide
- Files.Read.All (Delegated) = read files the signed-in user can already access
- Files.Read.All (Application) = read files in all site collections, tenant-wide

This distinction is easy to miss when quickly reviewing a permissions list, since the permission names themselves are identical.

3. Build

Three Microsoft Graph application permissions were added to the existing 'Expense Reporting Tool' app registration: Files.Read.All, Mail.Read, and User.Read.All. Unlike the delegated permissions used in the earlier consent-grant lab, all three application permissions show 'Admin consent required: Yes' by default; there is no user-consent path for application permissions at all. Only an administrator can grant them.

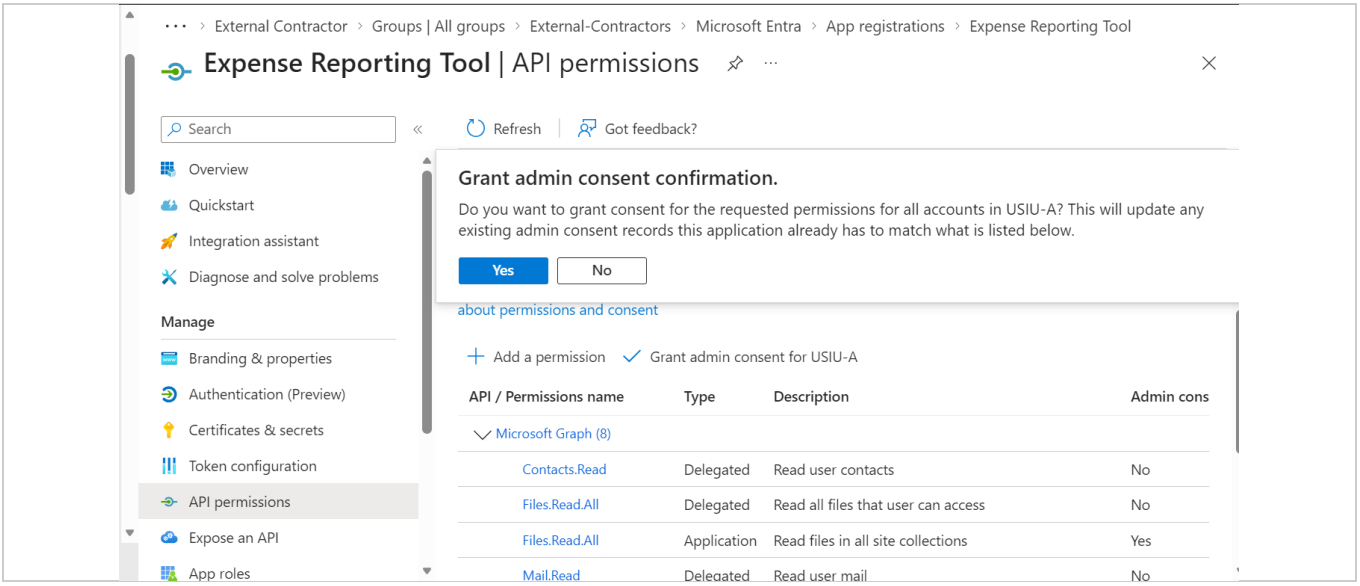


Figure 1 — Application-type permissions added, each requiring admin consent

Admin consent was granted for all accounts in the tenant with a single confirmation:

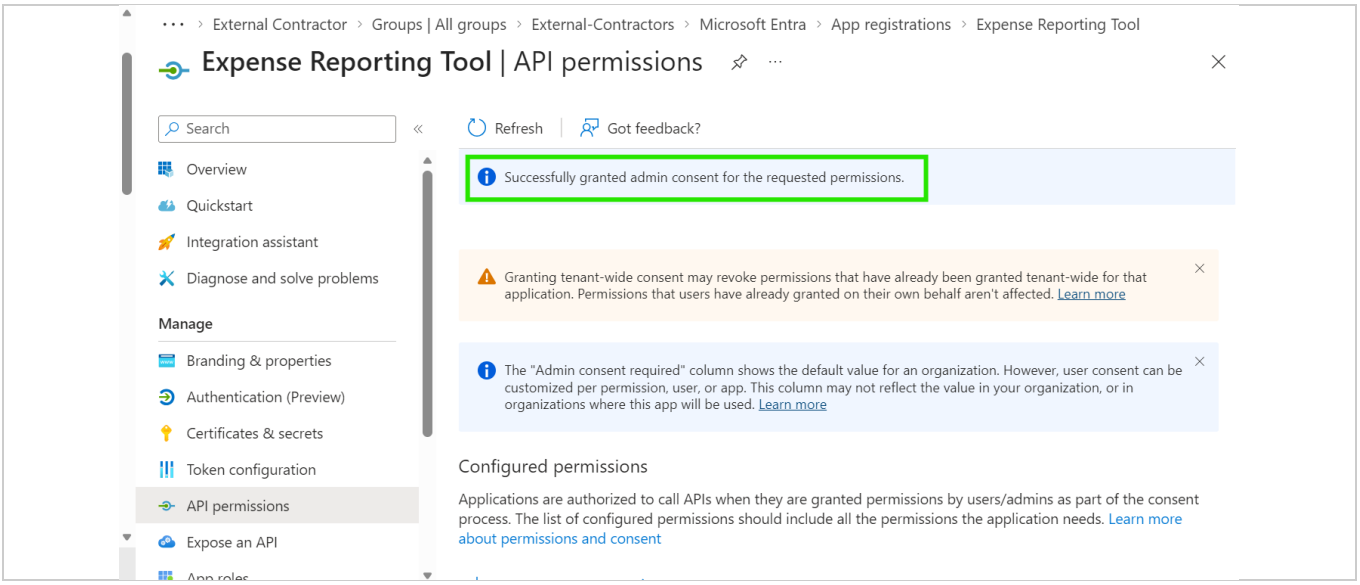


Figure 2 — "Successfully granted admin consent for the requested permissions"

4. Attack Narrative: Client Credentials Flow

With admin consent granted, the application authenticated directly using its own client ID and secret, via the OAuth2 client credentials grant. This flow requires no user interaction of any kind:

```
$body = @{
    client_id     = "<client-id>"
    client_secret = "<client-secret>"
    scope         = "https://graph.microsoft.com/.default"
    grant_type    = "client_credentials"
}
$response = Invoke-RestMethod -Method Post `
  -Uri "https://login.microsoftonline.com/<tenant>/oauth2/v2.0/token" -Body $body
$response.access_token
```

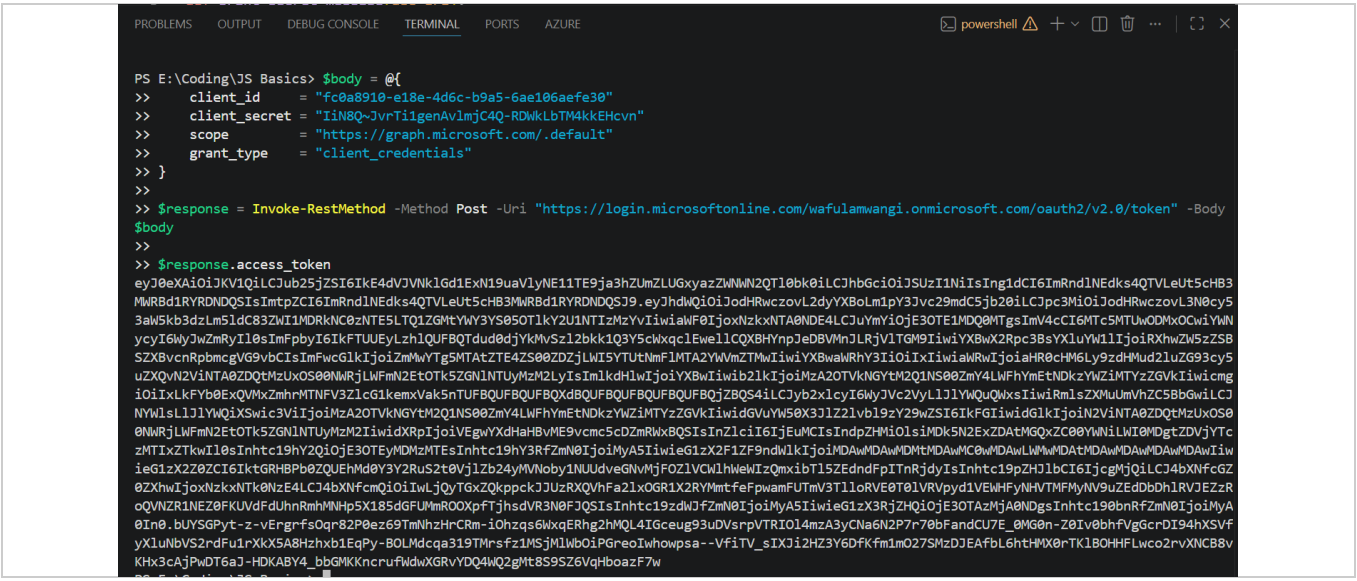


Figure 3 — Client credentials token request and raw access token returned

The resulting token was decoded via jwt.ms. Two claims confirm the nature of this access: idtyp: app explicitly marks this as an application-only (non-user) token, and the roles claim lists exactly the three granted permissions. User.Read.All, Files.Read.All, Mail.Read. Application tokens carry a roles claim in place of the scp claim seen in delegated tokens. No upn, unique_name, or name claim is present anywhere in the token, because there is no user.

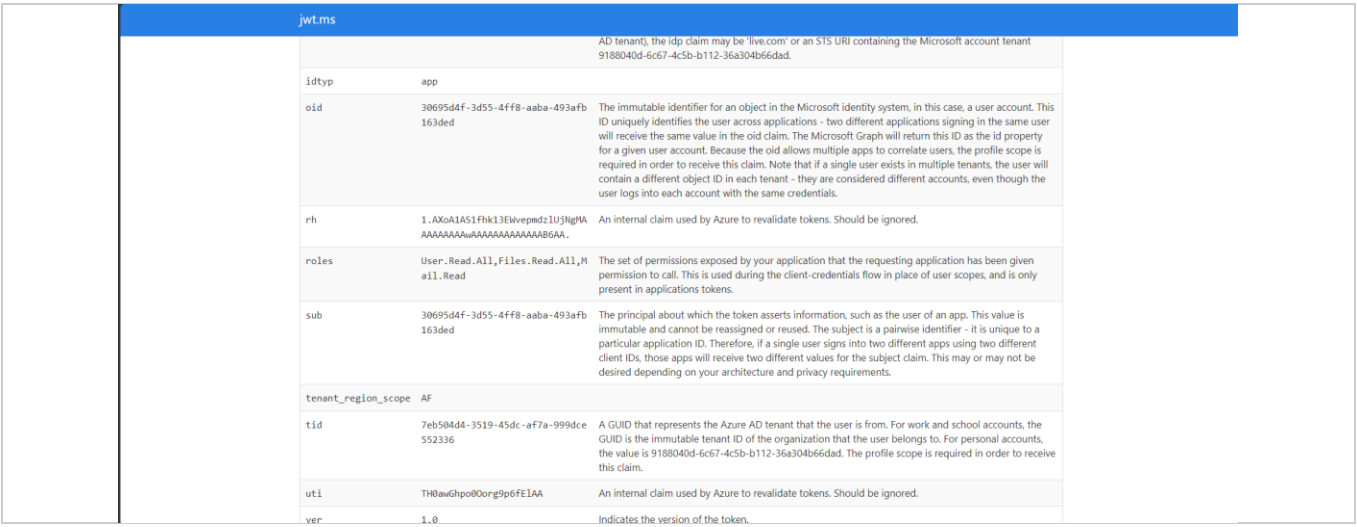


Figure 4 — Decoded token claims: idtyp: app, roles: User.Read.All, Files.Read.All, Mail.Read

5. Impact: Live Verification

To confirm the access was not merely theoretical, the token was used to call the Microsoft Graph /users endpoint directly:

```
$headers = @{ Authorization = "Bearer $($response.access_token)" }
Invoke-RestMethod -Headers $headers `
  -Uri "https://graph.microsoft.com/v1.0/users" |
  Select-Object -ExpandProperty value | Select-Object displayName, mail
```

The call succeeded immediately, returning every user account in the tenant with display name and email address, with no consent prompt, no session, and no user involved at any point in the request:

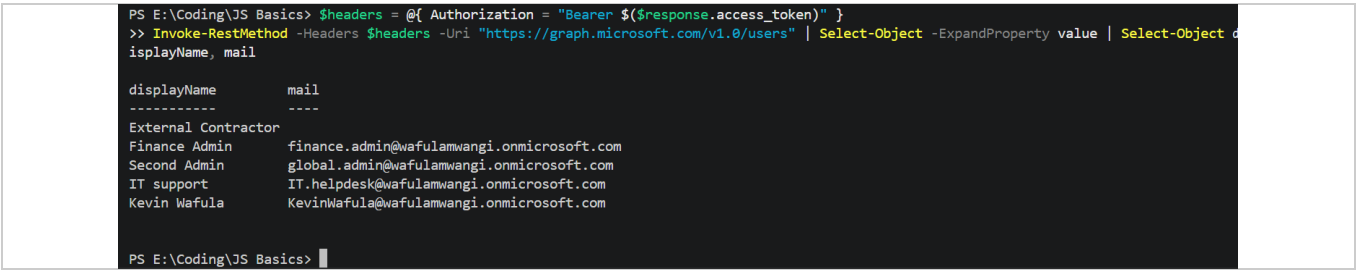


Figure 5 — Live Graph API call enumerating every user in the tenant via application-only token

The same token additionally carries unattended, tenant-wide read access to every mailbox (Mail.Read) and every SharePoint site collection (Files.Read.All) in the organization. Access that persists for as long as the client secret remains valid and the admin consent grant remains in place, with no re-authentication, re-consent, or user activity required at any point.

6. Detection

- Entra ID Audit Logs: alert on any 'Consent to application' event where consentType = AllPrincipals and the permission type is Application rather than Delegated. These warrant a higher review priority given the absence of any user-level checkpoint
- Entra ID Sign-in Logs → Service principal sign-ins: monitor client credentials grant activity for a given app registration, particularly spikes in frequency or calls to sensitive endpoints (/users, /messages, /drives) shortly after a new grant
- Microsoft Graph Activity Logs (where enabled): review which service principals are actively calling which Graph endpoints, and flag any application holding broad Application-type Mail/Files/User permissions it does not appear to use
- Periodically enumerate all app registrations with Application-type Graph permissions granted, independent of whether they are Delegated-only elsewhere, since this distinction is easy to miss in a routine permissions review

7. Remediation Recommendations

- Apply least privilege rigorously to application permissions, specifically request the narrowest Graph permission that satisfies the integration's actual need (e.g., Mail.Read for a single shared mailbox via application access policies, rather than tenant-wide Mail.Read)
- Use Exchange Online application access policies to scope Mail.Read (Application) down to specific mailboxes rather than leaving it tenant-wide by default
- Store client secrets in a managed secret store (Azure Key Vault or equivalent) with access logging, rather than embedding them in scripts or configuration files
- Prefer certificate-based authentication or managed identities over client secrets where the integration supports it, since secrets are more easily copied, leaked, or committed to source control by accident
- Set a short expiry on client secrets and enforce a rotation policy, rather than relying on long-lived or never-expiring secrets
- Review all Application-type Graph permission grants on a recurring schedule, treating each as a standing, unattended credential with the same scrutiny given to a privileged service account

8. Appendix — Evidence Index

- Figure 1 — Application permissions added, admin consent required
- Figure 2 — Admin consent successfully granted, tenant-wide
- Figure 3 — Client credentials token request (PowerShell)
- Figure 4 — Decoded token: idtyp: app, roles claim listing all three permissions
- Figure 5 — Live Graph API call enumerating every user in the tenant